VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Digital Signal Processing Laboratory

Laboratory Report 3

Discrete Time Signals and Systems

Advisor(s): Nguyễn Công Thái

Student(s): Vũ Đức Minh 2452774

Phương Quốc Khanh 2452488

HO CHI MINH CITY, OCTOBER 2026

Contents

1 Objectives and signal representation **6**

2 Exercise 1: Delay **6**
2.1 Task and method . . . 6
2.2 Source code . . . 7
2.3 Example and expected result . . . 8
2.4 Execution evidence . . . 8
2.5 Discussion . . . 9

3 Exercise 2: Advance **10**
3.1 Task and method . . . 10
3.2 Source code . . . 10
3.3 Example and expected result . . . 11
3.4 Execution evidence . . . 12
3.5 Discussion . . . 12

4 Exercise 3: Folding **13**
4.1 Task and method . . . 13
4.2 Source code . . . 13
4.3 Example and expected result . . . 14
4.4 Execution evidence . . . 15
4.5 Discussion . . . 15

5 Exercise 4: Addition **17**
5.1 Task and method . . . 17
5.2 Source code . . . 17
5.3 Example and expected result . . . 19
5.4 Execution evidence . . . 20
5.5 Discussion . . . 20

6 Exercise 5: Multiplication **21**
6.1 Task and method . . . 21
6.2 Source code . . . 21
6.3 Example and expected result . . . 23
6.4 Execution evidence . . . 24
6.5 Discussion . . . 24

7 Exercise 6: Convolution **25**
7.1 Task and method . . . 25
7.2 Source code . . . 25
7.3 Example and expected result . . . 27
7.4 Execution evidence . . . 28
7.5 Discussion . . . 28

8 Supplementary Problem 2.1 **29**
8.1 Part (a): Sample values . . . 29
8.2 Part (b): Changing the operation order . . . 29
8.2.1 Fold first, then delay by four samples . . . 29
8.2.2 Delay first, then fold . . . 30
8.3 Part (c): The signal x(-n+4) . . . 30
8.4 Part (d): General rule . . . 30
8.5 Part (e): Impulse and unit-step representations . . . 31
8.6 Scilab source and expected checks . . . 31
8.7 Execution evidence . . . 33

9 Supplementary Problem 2.2 **34**
9.1 Part (a): Delay by two samples . . . 34
9.2 Part (b): Folding followed by a four-sample delay . . . 34
9.3 Part (c): Advance by two samples . . . 34
9.4 Part (d): Multiplication by a reversed step . . . 35
9.5 Part (e): Multiplication by a shifted impulse . . . 35
9.6 Part (f): A squared time argument . . . 35
9.7 Parts (g) and (h): Even and odd components . . . 35
9.8 Scilab implementation . . . 36
9.9 Execution evidence . . . 38

10 Supplementary Problem 2.4 **40**
10.1 Existence of the decomposition . . . 40
10.2 Uniqueness of the decomposition . . . 40
10.3 Example and expected result . . . 41
10.4 Scilab implementation . . . 41
10.5 Execution evidence . . . 43



11 Supplementary Problem 2.6 **44**
11.1 Criterion for time invariance 44
11.2 Part (a): Squared-time system 44
11.3 Part (b): Explicit pulse example 44
11.4 Part (c): First-difference system 45
11.5 Part (d): Multiplication by time 46
11.6 Scilab implementation and expected results 47
11.7 Execution evidence 49

12 Conclusion **50**

1 Objectives and signal representation

This laboratory reviews discrete-time signals and systems through Scilab functions for time shifting, folding, addition, pointwise multiplication, and convolution. Each function is illustrated with an example, its source code, and graphical or numerical output.

A finite sequence is represented by a vector `xn` and an integer `xorigin`. If `xorigin` is within the stored vector, it is the position of the sample at $n = 0$. Since Scilab indices start at one, the time corresponding to vector position i is

$$n_i = i - \texttt{xorigin}, \qquad i = 1, \ldots, N, \tag{1.1}$$

where N is the vector length. Thus the time axis is constructed as

```
nx = (1:length(xn)) - xorigin;
```

Samples outside the stored interval are assumed to be zero. After a time shift, the origin parameter may fall outside $1, \ldots, N$; it still defines the time coordinates and must not be used directly as a vector index in that case.

For example, `xn = [1, -2, 3, 6]` and `xorigin = 3` represent $x(-2) = 1$, $x(-1) = -2$, $x(0) = 3$, and $x(1) = 6$.

The plots use vertical bars to represent discrete samples. Input and output plots share the same axis limits, with a line thickness of three pixels, so that their time positions can be compared directly.

2 Exercise 1: Delay

2.1 Task and method

Implement `[yn, yorigin] = delay(xn, xorigin, k)` for

$$y(n) = x(n - k), \qquad k > 0, \tag{2.1}$$

where k is an integer number of samples. A sample originally at n_i appears at $n_i + k$ in the output. The vector values and their order are unchanged. The output origin follows from

$$i - \texttt{yorigin} = i - \texttt{xorigin} + k, \tag{2.2}$$

$$\texttt{yorigin} = \texttt{xorigin} - k. \tag{2.3}$$

The implementation constructs the two time axes separately and displays the signals in two subplots within one figure.

2.2 Source code

```
1
2  function [yn, yorigin] = delay(xn, xorigin, k)
3      yn = xn;
4      yorigin = xorigin - k;
5
6      nx = (1:length(xn)) - xorigin;
7      ny = (1:length(yn)) - yorigin;
8
9      nmin = min([nx, ny]) - 1;
10     nmax = max([nx, ny]) + 1;
11     ymin = min([0, xn, yn]) - 1;
12     ymax = max([0, xn, yn]) + 1;
13
14     scf();
15     subplot(2, 1, 1);
16     plot2d3(nx, xn);
17     e = gce();
18     e.children(1).thickness = 3;
19     a = gca();
20     a.data_bounds = [nmin, ymin; nmax, ymax];
21     xtitle("x(n)", "n", "A");
22     xgrid();
23
24     subplot(2, 1, 2);
25     plot2d3(ny, yn);
26     e = gce();
27     e.children(1).thickness = 3;
28     a = gca();
29     a.data_bounds = [nmin, ymin; nmax, ymax];
30     xtitle("y(n)", "n", "A");
31     xgrid();
32 endfunction
33
34 [yn, yorigin] = delay([1, -2, 3, 6], 3, 1);
35
36 disp("_yn =");
37 disp(yn);
```

```
disp("y_o =");
disp(yorigin);
```

Listing 2.1: Scilab delay function and example call

2.3 Example and expected result

For the example in the lab handout,

```
[yn, yorigin] = delay([1, -2, 3, 6], 3, 1);
```

the calculated output is

$$\texttt{yn} = [1, -2, 3, 6], \qquad \texttt{yorigin} = 2.$$

Table 2.1: Sample coordinates before and after a one-sample delay

Vector position i	1	2	3	4
Sample value	1	-2	3	6
Input time n_i	-2	-1	0	1
Output time $n_i + 1$	-1	0	1	2

2.4 Execution evidence

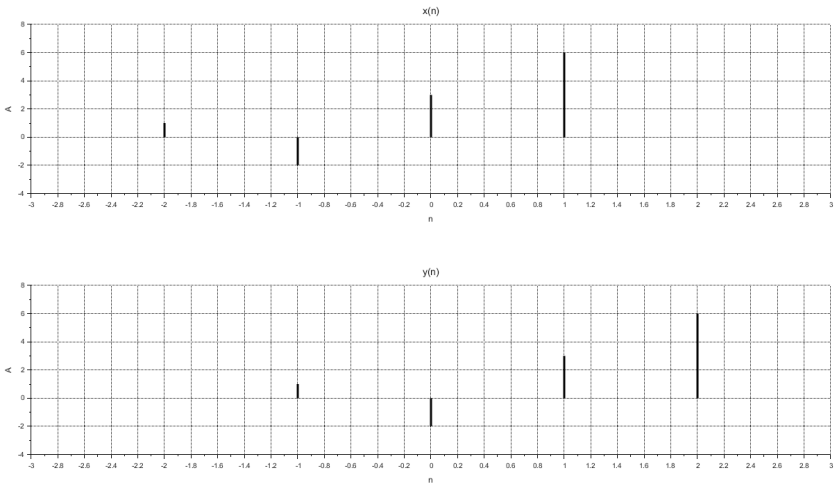


Figure 2.1: Input and delayed output for Exercise 1



The captured Scilab plot agrees with the calculated sample coordinates in Table 2.1; the output subplot contains only the delayed signal.

2.5 Discussion

The captured output graph is shifted to the right by one sample. In particular, the sample of amplitude 3 moves from $n = 0$ to $n = 1$, while the sample of amplitude 6 moves from $n = 1$ to $n = 2$. The output value at $n = 0$ is -2, which occupies vector position two. This explains why the origin parameter decreases even though the signal moves to the right.

3 Exercise 2: Advance

3.1 Task and method

Implement `[yn, yorigin] = advance(xn, xorigin, k)` for

$$y(n) = x(n + k), \qquad k > 0, \tag{3.1}$$

where k is an integer number of samples. Advancing the signal moves each sample to the left by k time units. A sample originally at n_i therefore appears at $n_i - k$. As with delay, the stored amplitudes and their order remain unchanged:

$$\texttt{yn} = \texttt{xn}, \tag{3.2}$$

$$i - \texttt{yorigin} = i - \texttt{xorigin} - k, \tag{3.3}$$

$$\texttt{yorigin} = \texttt{xorigin} + k. \tag{3.4}$$

The origin parameter increases because it is a reference vector position, not the direction of the time shift.

3.2 Source code

```
// Exercise 2: advance a discrete-time signal by k samples (k > 0).
function [yn, yorigin] = advance(xn, xorigin, k)
    yn = xn;
    yorigin = xorigin + k;

    nx = (1:length(xn)) - xorigin;
    ny = (1:length(yn)) - yorigin;

    nmin = min([nx, ny]) - 1;
    nmax = max([nx, ny]) + 1;
    ymin = min([0, xn, yn]) - 1;
    ymax = max([0, xn, yn]) + 1;

    scf();
    subplot(2, 1, 1);
    plot2d3(nx, xn);
    e = gce();
    e.children(1).thickness = 3;
    a = gca();
```

```
    a.data_bounds = [nmin, ymin; nmax, ymax];
    xtitle("x(n)", "n", "A");
    xgrid();

    subplot(2, 1, 2);
    plot2d3(ny, yn);
    e = gce();
    e.children(1).thickness = 3;
    a = gca();
    a.data_bounds = [nmin, ymin; nmax, ymax];
    xtitle("y(n) = x(n+k)", "n", "A");
    xgrid();
endfunction

// Example: advance the input signal by one sample.
[yn, yorigin] = advance([1, -2, 3, 6], 3, 1);

disp("yn =");
disp(yn);
disp("yorigin =");
disp(yorigin);
```

Listing 3.1: Scilab advance function and example call

3.3 Example and expected result

Use the same input sequence as in Exercise 1 to compare the two operations:

```
[yn, yorigin] = advance([1, -2, 3, 6], 3, 1);
```

The calculated output is

$$\texttt{yn} = [1, -2, 3, 6], \qquad \texttt{yorigin} = 4.$$

Table 3.1: Sample coordinates before and after a one-sample advance

Vector position i	1	2	3	4
Sample value	1	-2	3	6
Input time n_i	-2	-1	0	1
Output time $n_i - 1$	-3	-2	-1	0

3.4 Execution evidence

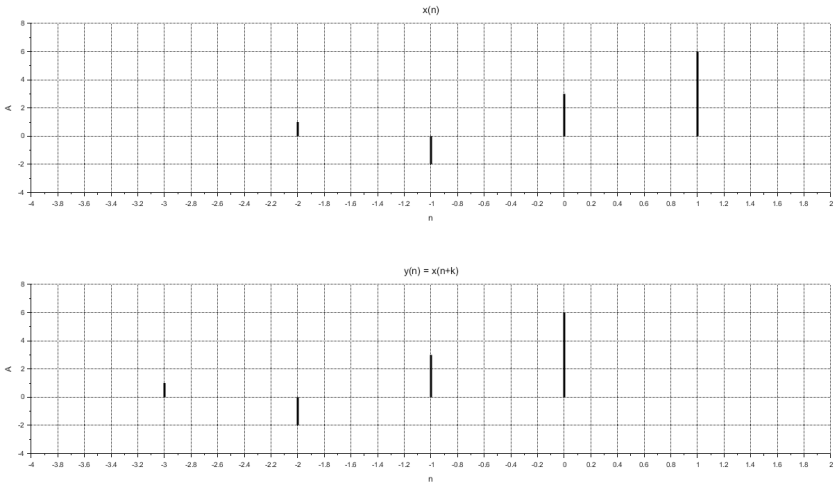


Figure 3.1: Input and advanced output for Exercise 2

The captured Scilab plot agrees with the calculated sample coordinates in Table 3.1.

3.5 Discussion

The captured output graph moves to the left by one sample without changing any amplitude. The sample of amplitude 3 moves from $n = 0$ to $n = -1$, and the sample of amplitude 6 moves from $n = 1$ to $n = 0$. Since the latter is the fourth element of the vector, the output origin is four. Delay and advance therefore use opposite changes to the origin parameter: subtract k for delay and add k for advance.

4 Exercise 3: Folding

4.1 Task and method

Implement `[yn, yorigin] = fold(xn, xorigin)` for

$$y(n) = x(-n). \tag{4.1}$$

Folding reflects the signal about $n = 0$. Each sample at time n_i moves to $-n_i$ while retaining its amplitude. To keep the output vector ordered by increasing time, its elements must be stored in reverse order.

Let N be the input vector length. Input position i becomes output position $j = N + 1 - i$. The corresponding time relation is

$$j - \texttt{yorigin} = -(i - \texttt{xorigin}), \tag{4.2}$$

$$N + 1 - i - \texttt{yorigin} = -i + \texttt{xorigin}, \tag{4.3}$$

$$\texttt{yorigin} = N + 1 - \texttt{xorigin}. \tag{4.4}$$

In Scilab, `xn($:-1:1)` selects the elements from the last position to the first, using a step of -1. Both the vector order and the origin parameter must be updated.

4.2 Source code

```
// Exercise 3: fold a discrete-time signal about n = 0.
function [yn, yorigin] = fold(xn, xorigin)
    yn = xn($:-1:1);
    yorigin = length(xn) + 1 - xorigin;

    nx = (1:length(xn)) - xorigin;
    ny = (1:length(yn)) - yorigin;

    nmin = min([nx, ny]) - 1;
    nmax = max([nx, ny]) + 1;
    ymin = min([0, xn, yn]) - 1;
    ymax = max([0, xn, yn]) + 1;

    scf();
    subplot(2, 1, 1);
    plot2d3(nx, xn);
    e = gce();
```

```
    e.children(1).thickness = 3;
    a = gca();
    a.data_bounds = [nmin, ymin; nmax, ymax];
    xtitle("x(n)", "n", "A");
    xgrid();

    subplot(2, 1, 2);
    plot2d3(ny, yn);
    e = gce();
    e.children(1).thickness = 3;
    a = gca();
    a.data_bounds = [nmin, ymin; nmax, ymax];
    xtitle("y(n) = x(-n)", "n", "A");
    xgrid();
endfunction

// Example: reverse the time coordinates of the input signal.
[yn, yorigin] = fold([1, -2, 3, 6], 3);

disp("yn =");
disp(yn);
disp("yorigin =");
disp(yorigin);
```

Listing 4.1: Scilab folding function and example call

4.3 Example and expected result

Use the same input sequence as in the preceding exercises:

```
[yn, yorigin] = fold([1, -2, 3, 6], 3);
```

The calculated output is

$$\texttt{yn} = [6, 3, -2, 1], \qquad \texttt{yorigin} = 4 + 1 - 3 = 2.$$

Table 4.1: Time reversal of each input sample

Sample value	1	−2	3	6
Input time n_i	−2	−1	0	1
Reflected time $-n_i$	2	1	0	−1

Reading the reflected samples in increasing time gives

Table 4.2: Output vector arranged in increasing time

Output vector position	1	2	3	4
Output time n	−1	0	1	2
Output value $y(n)$	6	3	−2	1

4.4 Execution evidence

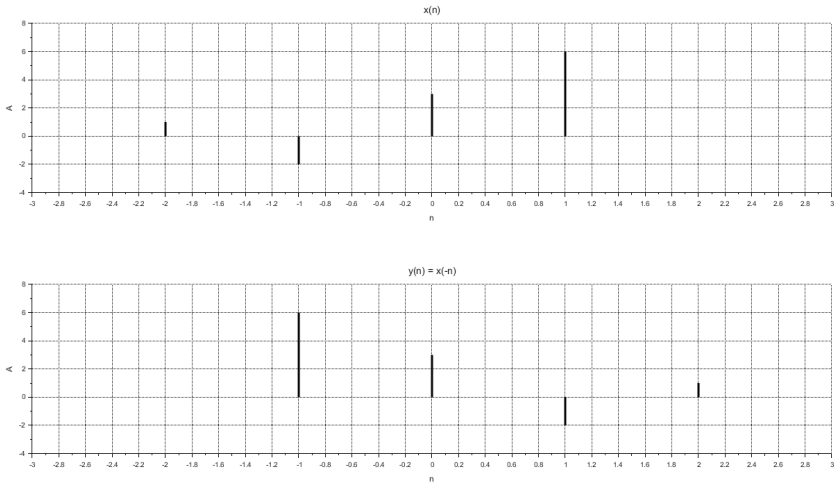


Figure 4.1: Input and folded output for Exercise 3

The captured Scilab plot agrees with the calculated output coordinates in Table 4.2.

4.5 Discussion

The captured output is the reflection of the input about the vertical axis at $n = 0$. The sample of amplitude 6 moves from $n = 1$ to $n = -1$, and the sample of amplitude 1 moves

from $n = -2$ to $n = 2$. The sample at $n = 0$ retains its amplitude 3, although its position within the stored vector changes from three to two. Folding changes time coordinates, not the signs of the amplitudes.



5 Exercise 4: Addition

5.1 Task and method

Implement `[yn, yorigin] = add(x1n, x1origin, x2n, x2origin)` to obtain

$$y(n) = x_1(n) + x_2(n). \tag{5.1}$$

The samples being added must refer to the same time n. Equal vector lengths alone do not ensure that the samples are aligned, since the two input origin positions may differ.

Let N_1, N_2 denote the vector lengths and o_1, o_2 their origin parameters. The stored time intervals are

$$1 - o_1 \le n \le N_1 - o_1, \qquad 1 - o_2 \le n \le N_2 - o_2.$$

Construct a common interval with endpoints

$$n_{\min} = \min(1 - o_1, 1 - o_2), \tag{5.2}$$
$$n_{\max} = \max(N_1 - o_1, N_2 - o_2). \tag{5.3}$$

Initialize two vectors of length $n_{\max} - n_{\min} + 1$ with zeros. Place each input sample at its common-vector position

$$i = n - n_{\min} + 1. \tag{5.4}$$

Samples outside each input's stored interval remain zero. The aligned vectors can then be added element by element. The output origin is

$$\texttt{yorigin} = 1 - n_{\min}. \tag{5.5}$$

The implementation displays both aligned inputs and their sum in three subplots within the same figure, using shared axis limits.

5.2 Source code

```
// Exercise 4: add two signals after aligning their time coordinates.
function [yn, yorigin] = add(x1n, x1origin, x2n, x2origin)
    // Use row vectors for both inputs.
    x1n = matrix(x1n, 1, -1);
    x2n = matrix(x2n, 1, -1);
```

```
6
7   n1 = (1:length(x1n)) - x1origin;
8   n2 = (1:length(x2n)) - x2origin;
9
10  // Common time interval covering both input vectors.
11  nfirst = min([n1(1), n2(1)]);
12  nlast = max([n1($), n2($)]);
13  ny = nfirst:nlast;
14  yorigin = 1 - nfirst;
15
16  // Insert each input at the positions for its own time coordinates.
17  x1aligned = zeros(1, length(ny));
18  x2aligned = zeros(1, length(ny));
19  x1aligned(n1 - nfirst + 1) = x1n;
20  x2aligned(n2 - nfirst + 1) = x2n;
21
22  yn = x1aligned + x2aligned;
23
24  // Three subplots with the same axis limits and line thickness.
25  ymin = min([0, x1aligned, x2aligned, yn]) - 1;
26  ymax = max([0, x1aligned, x2aligned, yn]) + 1;
27
28  scf();
29  subplot(3, 1, 1);
30  plot2d3(ny, x1aligned);
31  e = gce();
32  e.children(1).thickness = 3;
33  a = gca();
34  a.data_bounds = [nfirst-1, ymin; nlast+1, ymax];
35  xtitle("x1(n)", "n", "A");
36  xgrid();
37
38  subplot(3, 1, 2);
39  plot2d3(ny, x2aligned);
40  e = gce();
41  e.children(1).thickness = 3;
42  a = gca();
43  a.data_bounds = [nfirst-1, ymin; nlast+1, ymax];
44  xtitle("x2(n)", "n", "A");
45  xgrid();
46
47  subplot(3, 1, 3);
48  plot2d3(ny, yn);
```

```
    e = gce();
    e.children(1).thickness = 3;
    a = gca();
    a.data_bounds = [nfirst-1, ymin; nlast+1, ymax];
    xtitle("y(n) = x1(n) + x2(n)", "n", "A");
    xgrid();
endfunction

// Example from the lab handout.
[yn, yorigin] = add([0, 1, 3, -2], 1, [1, 1, 2, 3], 2);

disp("yn =");
disp(yn);
disp("yorigin =");
disp(yorigin);
```

Listing 5.1: Scilab addition function and example call

5.3 Example and expected result

The handout specifies the example

```
[yn, yorigin] = add([0, 1, 3, -2], 1, [1, 1, 2, 3], 2);
```

The first vector covers $n = 0, 1, 2, 3$, whereas the second covers $n = -1, 0, 1, 2$. The common time interval is therefore $-1 \le n \le 3$.

Table 5.1: Time alignment and samplewise addition

Time n	-1	0	1	2	3
Aligned $x_1(n)$	0	0	1	3	-2
Aligned $x_2(n)$	1	1	2	3	0
Sum $y(n)$	1	1	3	6	-2

The calculated output agrees with the result stated in the handout:

$$\texttt{yn} = [1, 1, 3, 6, -2], \qquad \texttt{yorigin} = 2.$$

5.4 Execution evidence

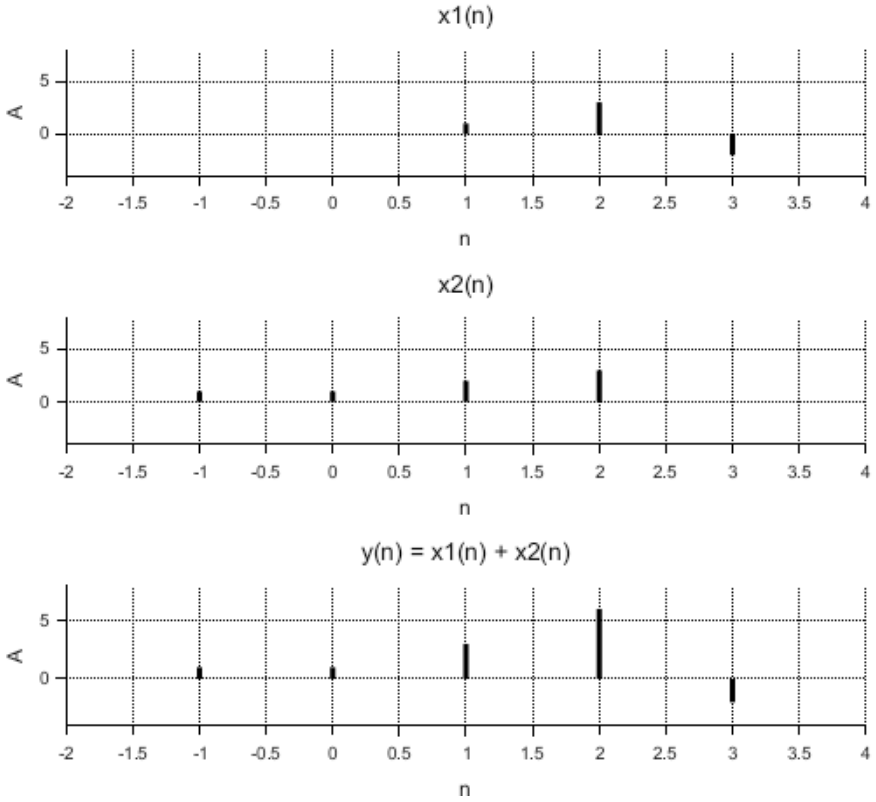


Figure 5.1: Aligned input signals and their sum for Exercise 4

The captured Scilab plot agrees with the time-aligned sum in Table 5.1, including the output samples at both ends of the interval.

5.5 Discussion

At $n = 2$, the two samples have amplitudes 3 and 3, giving an output of 6. At $n = -1$, only the second input contributes, so $y(-1) = 1$. At $n = 3$, only the first input contributes, so $y(3) = -2$. The output contains five stored samples even though each input contains four. Directly adding the original vectors would incorrectly pair samples at different time indices and would not represent the required signal sum.

6 Exercise 5: Multiplication

6.1 Task and method

Implement `[yn, yorigin] = multi(x1n, x1origin, x2n, x2origin)` to calculate the pointwise product

$$y(n) = x_1(n)x_2(n). \tag{6.1}$$

As in Exercise 4, the input samples must first be aligned by time. Let N_1, N_2 be the input vector lengths and o_1, o_2 their origins. The implementation retains the full common interval

$$n_{\min} = \min(1 - o_1, 1 - o_2), \tag{6.2}$$

$$n_{\max} = \max(N_1 - o_1, N_2 - o_2). \tag{6.3}$$

Two zero-filled vectors are created on this interval. Each stored input sample at time n is inserted at position $n - n_{\min} + 1$. The output is then obtained with the Scilab elementwise operator:

```
yn = x1aligned .* x2aligned;
yorigin = 1 - nfirst;
```

The operator `.*` multiplies corresponding samples; `*` performs matrix multiplication. The pointwise product is also distinct from the convolution operation in Exercise 6.

6.2 Source code

```
// Exercise 5: multiply two signals at matching time coordinates.
function [yn, yorigin] = multi(x1n, x1origin, x2n, x2origin)
    x1n = matrix(x1n, 1, -1);
    x2n = matrix(x2n, 1, -1);

    n1 = (1:length(x1n)) - x1origin;
    n2 = (1:length(x2n)) - x2origin;

    // Keep the full common interval, including zero-valued samples.
    nfirst = min([n1(1), n2(1)]);
    nlast = max([n1($), n2($)]);
    ny = nfirst:nlast;
    yorigin = 1 - nfirst;

```

```
    x1aligned = zeros(1, length(ny));
    x2aligned = zeros(1, length(ny));
    x1aligned(n1 - nfirst + 1) = x1n;
    x2aligned(n2 - nfirst + 1) = x2n;

    // Elementwise multiplication, not matrix multiplication.
    yn = x1aligned .* x2aligned;

    ymin = min([0, x1aligned, x2aligned, yn]) - 1;
    ymax = max([0, x1aligned, x2aligned, yn]) + 1;

    scf();
    subplot(3, 1, 1);
    plot2d3(ny, x1aligned);
    e = gce();
    e.children(1).thickness = 3;
    a = gca();
    a.data_bounds = [nfirst-1, ymin; nlast+1, ymax];
    xtitle("x1(n)", "n", "A");
    xgrid();

    subplot(3, 1, 2);
    plot2d3(ny, x2aligned);
    e = gce();
    e.children(1).thickness = 3;
    a = gca();
    a.data_bounds = [nfirst-1, ymin; nlast+1, ymax];
    xtitle("x2(n)", "n", "A");
    xgrid();

    subplot(3, 1, 3);
    plot2d3(ny, yn);
    e = gce();
    e.children(1).thickness = 3;
    a = gca();
    a.data_bounds = [nfirst-1, ymin; nlast+1, ymax];
    xtitle("y(n) = x1(n) . x2(n)", "n", "A");
    xgrid();
endfunction

// Reuse the inputs from Exercise 4 to compare sum and product.
[yn, yorigin] = multi([0, 1, 3, -2], 1, [1, 1, 2, 3], 2);

```

```
disp("yn =");
disp(yn);
disp("yorigin =");
disp(yorigin);
```

Listing 6.1: Scilab pointwise multiplication function and example call

6.3 Example and expected result

Reuse the two input signals from Exercise 4:

```
[yn, yorigin] = multi([0, 1, 3, -2], 1, [1, 1, 2, 3], 2);
```

Both inputs are represented on $n = -1, 0, 1, 2, 3$ before multiplication.

Table 6.1: Time alignment and pointwise multiplication

Time n	-1	0	1	2	3
Aligned $x_1(n)$	0	0	1	3	-2
Aligned $x_2(n)$	1	1	2	3	0
Product $y(n)$	0	0	2	9	0

The calculated output is

$$\texttt{yn} = [0, 0, 2, 9, 0], \qquad \texttt{yorigin} = 2.$$

Zero-valued samples at the interval boundaries are retained. The origin therefore refers to the second position, even though the first nonzero output occurs at the third position.

6.4 Execution evidence

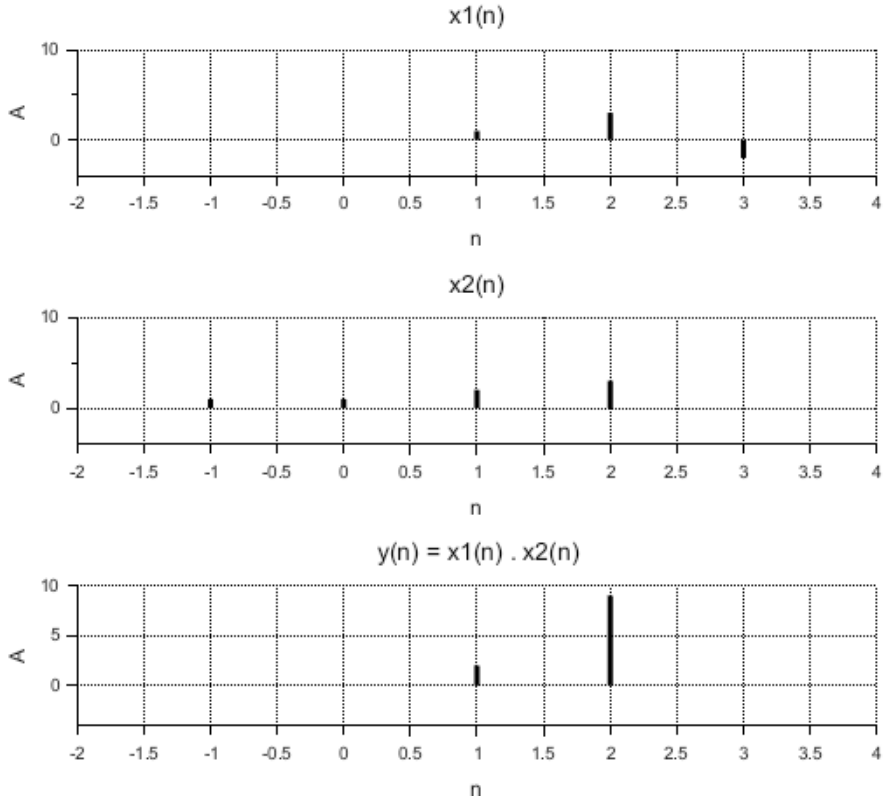


Figure 6.1: Aligned input signals and their pointwise product for Exercise 5

The captured Scilab plot agrees with Table 6.1: the output has two nonzero samples, at $n = 1$ and $n = 2$.

6.5 Discussion

At $n = 1$, the product is $1 \times 2 = 2$, and at $n = 2$ it is $3 \times 3 = 9$. At $n = -1$, $n = 0$, and $n = 3$, at least one factor is zero, so the output is zero. The captured output plot therefore contains two nonzero bars, of amplitudes 2 and 9 at $n = 1$ and $n = 2$, respectively. In general, a pointwise product can be nonzero only where both input signals are nonzero.



7 Exercise 6: Convolution

7.1 Task and method

Implement `[yn, yorigin] = convolution(xn, xorigin, hn, horigin)` to calculate

$$y(n) = (x * h)(n) = \sum_{k=-\infty}^{\infty} x(k)h(n-k). \tag{7.1}$$

For an LTI system with impulse response $h(n)$ and input $x(n)$, this convolution gives the output. Only finitely many terms contribute for the stored finite sequences, which are zero outside their intervals.

Let the input lengths be N and M. Initialize an output vector with $N + M - 1$ zeros. For each pair of input vector positions i and j, accumulate its contribution at output position $i + j - 1$:

```
for i = 1:N
    for j = 1:M
        yn(i+j-1) = yn(i+j-1) + xn(i)*hn(j);
    end
end
```

The product contributes at the sum of the input sample times,

$$n = (i - \texttt{xorigin}) + (j - \texttt{horigin}). \tag{7.2}$$

At output vector position $r = i + j - 1$, the time is also $n = r - \texttt{yorigin}$. Equating these expressions gives

$$\texttt{yorigin} = \texttt{xorigin} + \texttt{horigin} - 1. \tag{7.3}$$

Thus the output interval runs from $2-\texttt{xorigin}-\texttt{horigin}$ to $N+M-\texttt{xorigin}-\texttt{horigin}$. The implementation also plots $x(n)$, $h(n)$, and $y(n)$ in a single figure.

7.2 Source code

```
1 // Exercise 6: convolution of two finite discrete-time signals.
2 function [yn, yorigin] = convolution(xn, xorigin, hn, horigin)
3     xn = matrix(xn, 1, -1);
4     hn = matrix(hn, 1, -1);
5
6     N = length(xn);
```

```
    M = length(hn);
    yn = zeros(1, N + M - 1);

    // Each pair contributes to the output at the sum of its times.
    for i = 1:N
        for j = 1:M
            yn(i+j-1) = yn(i+j-1) + xn(i)*hn(j);
        end
    end
    yorigin = xorigin + horigin - 1;

    nx = (1:N) - xorigin;
    nh = (1:M) - horigin;
    ny = (1:length(yn)) - yorigin;

    nmin = min([nx, nh, ny]) - 1;
    nmax = max([nx, nh, ny]) + 1;
    ymin = min([0, xn, hn, yn]) - 1;
    ymax = max([0, xn, hn, yn]) + 1;

    scf();
    subplot(3, 1, 1);
    plot2d3(nx, xn);
    e = gce();
    e.children(1).thickness = 3;
    a = gca();
    a.data_bounds = [nmin, ymin; nmax, ymax];
    xtitle("x(n)", "n", "A");
    xgrid();

    subplot(3, 1, 2);
    plot2d3(nh, hn);
    e = gce();
    e.children(1).thickness = 3;
    a = gca();
    a.data_bounds = [nmin, ymin; nmax, ymax];
    xtitle("h(n)", "n", "A");
    xgrid();

    subplot(3, 1, 3);
    plot2d3(ny, yn);
    e = gce();
    e.children(1).thickness = 3;
```

```
    a = gca();
    a.data_bounds = [nmin, ymin; nmax, ymax];
    xtitle("y(n) = convolution of x(n) and h(n)", "n", "A");
    xgrid();
endfunction

// h(0) = h(1) = 1, so y(n) = x(n) + x(n-1).
[yn, yorigin] = convolution([1, -2, 3, 6], 3, [1, 1], 1);

disp("yn =");
disp(yn);
disp("yorigin =");
disp(yorigin);
```

Listing 7.1: Scilab convolution function and example call

7.3 Example and expected result

Choose the signal from Exercises 1–3 and a two-sample impulse response:

```
[yn, yorigin] = convolution([1, -2, 3, 6], 3, [1, 1], 1);
```

Here $x(n)$ is stored on $-2 \le n \le 1$ and $h(n) = \delta(n) + \delta(n-1)$. Substituting this impulse response into the convolution sum gives a direct check:

$$y(n) = x(n) + x(n-1). \tag{7.4}$$

Table 7.1: Manual calculation of the convolution example

Time n	-2	-1	0	1	2
$x(n)$	1	-2	3	6	0
$x(n-1)$	0	1	-2	3	6
$y(n)$	1	-1	1	9	6

The calculated result is

$$\texttt{yn} = [1, -1, 1, 9, 6], \qquad \texttt{yorigin} = 3 + 1 - 1 = 3.$$

The output has $4 + 2 - 1 = 5$ stored samples. Its third element corresponds to $n = 0$.

7.4 Execution evidence

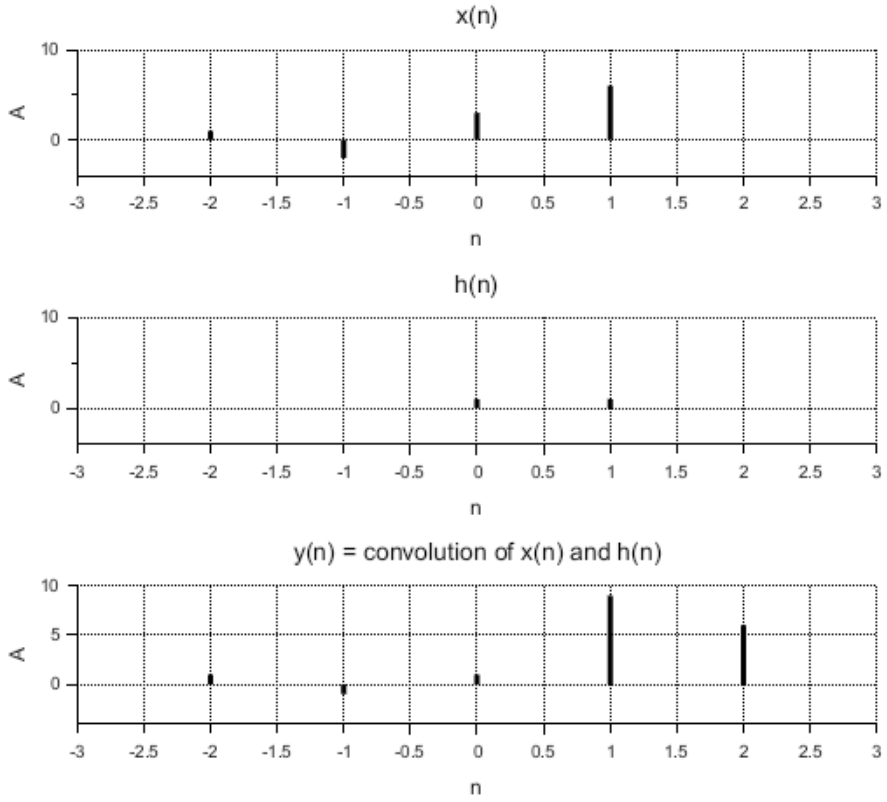


Figure 7.1: Input, impulse response, and convolution output for Exercise 6

The captured Scilab plot agrees with Table 7.1: the output samples are $1, -1, 1, 9, 6$ at $n = -2, -1, 0, 1, 2$, respectively.

7.5 Discussion

Each output sample is a sum of products at the appropriate time offsets. For this impulse response, the system adds the current input sample and the preceding input sample. In particular, $y(1) = 6 + 3 = 9$ and $y(2) = 0 + 6 = 6$. The output therefore extends one sample beyond the stored input interval. The zero padding used to describe the finite sequences must be respected when calculating these boundary samples.

8 Supplementary Problem 2.1

The reference problem defines the discrete-time signal

$$x(n) = \begin{cases} 1 + \dfrac{n}{3}, & -3 \le n \le -1, \\ 1, & 0 \le n \le 3, \\ 0, & \text{otherwise.} \end{cases} \tag{8.1}$$

All time indices in this problem are integers.

8.1 Part (a): Sample values

Substituting the integer time indices gives

Table 8.1: Sample values of the piecewise signal

n	-3	-2	-1	0	1	2	3
$x(n)$	0	$1/3$	$2/3$	1	1	1	1

The sample at $n = -3$ is zero, since $1 + (-3)/3 = 0$. One vector representation is

$$\texttt{xn} = [0, 1/3, 2/3, 1, 1, 1, 1], \qquad \texttt{xorigin} = 4.$$

8.2 Part (b): Changing the operation order

8.2.1 Fold first, then delay by four samples

Let $v(n) = x(-n)$. Delaying this intermediate signal gives

$$y_{b1}(n) = v(n-4) = x(-(n-4)) = x(4-n). \tag{8.2}$$

Equivalently, an original sample at time m moves first to $-m$ and then to $4 - m$. The resulting piecewise expression is

$$y_{b1}(n) = \begin{cases} 1, & 1 \le n \le 4, \\ (7-n)/3, & 5 \le n \le 7, \\ 0, & \text{otherwise.} \end{cases} \tag{8.3}$$

8.2.2 Delay first, then fold

Let $w(n) = x(n-4)$. Folding this intermediate signal gives

$$y_{b2}(n) = w(-n) = x(-n-4). \tag{8.4}$$

An original sample at time m moves first to $m+4$ and then to $-m-4$. Therefore,

$$y_{b2}(n) = \begin{cases} 1, & -7 \le n \le -4, \\ (-n-1)/3, & -3 \le n \le -1, \\ 0, & \text{otherwise.} \end{cases} \tag{8.5}$$

Table 8.2: Transformed samples for the two operation orders

Time for y_{b1}	1	2	3	4	5	6	7
Time for y_{b2}	-7	-6	-5	-4	-3	-2	-1
Corresponding value	1	1	1	1	2/3	1/3	0

8.3 Part (c): The signal x(-n+4)

Since $-n+4 = 4-n$, the required signal is exactly $y_{b1}(n)$. Its plot and values coincide with the fold-then-delay result in part (b).

8.4 Part (d): General rule

For $y(n) = x(-n+k) = x(-(n-k))$, fold the original signal and then shift the folded signal to the right by k samples when $k > 0$. Equivalently, advance the original signal by k samples and then fold. The direct sample mapping is

$$m \longmapsto k - m. \tag{8.6}$$

For negative k, the shift directions reverse. Folding and a nonzero time shift do not commute in general; changing their order reverses the direction of the resulting shift. Here the original sample at $m = 0$ lands at $n = 4$ in y_{b1} but at $n = -4$ in y_{b2}.

8.5 Part (e): Impulse and unit-step representations

Define $\delta(n) = 1$ for $n = 0$ and zero otherwise, and $u(n) = 1$ for $n \geq 0$ and zero otherwise. Using a weighted impulse at each nonzero sample gives

$$x(n) = \tfrac{1}{3}\delta(n+2) + \tfrac{2}{3}\delta(n+1) + \delta(n) + \delta(n-1) + \delta(n-2) + \delta(n-3). \tag{8.7}$$

The two piecewise intervals can also be selected by differences of unit steps:

$$x(n) = \left(1 + \frac{n}{3}\right)\left[u(n+3) - u(n)\right] + u(n) - u(n-4). \tag{8.8}$$

The first step difference selects $-3 \leq n \leq -1$, and the second selects $0 \leq n \leq 3$, reproducing the original definition.

8.6 Scilab source and expected checks

The script evaluates the original piecewise function at each transformed time argument. It plots parts (a), (b1), (b2), and (c), and compares the two reconstructions from part (e) against the original samples on $-8 \leq n \leq 8$. This interval includes all nonzero samples of all plotted signals. Parts (b1) and (c) should agree exactly; reconstruction errors should be zero up to floating-point rounding.

```
// Supplementary Problem 2.1: piecewise signal, folding and shifting.
function values = signal21(n)
    values = zeros(1, length(n));
    ramp = find((n >= -3) & (n <= -1));
    flat = find((n >= 0) & (n <= 3));
    values(ramp) = 1 + n(ramp)/3;
    values(flat) = 1;
endfunction

function values = step21(n)
    values = bool2s(n >= 0);
endfunction

function values = impulse21(n)
    values = bool2s(n == 0);
endfunction

// Include all original and transformed samples, with zero padding.
```

```
n = -8:8;
xn = signal21(n);

// (b1) v(n) = x(-n), then y(n) = v(n-4) = x(-(n-4)).
yb1 = signal21(-(n-4));

// (b2) v(n) = x(n-4), then y(n) = v(-n) = x(-n-4).
yb2 = signal21(-n-4);

// (c) Direct evaluation of x(-n+4).
yc = signal21(-n+4);

// (e) Reconstruction from impulses and from unit steps.
xdelta = (1/3)*impulse21(n+2) + (2/3)*impulse21(n+1) ..
       + impulse21(n) + impulse21(n-1) ..
       + impulse21(n-2) + impulse21(n-3);
xstep = (1+n/3).*(step21(n+3)-step21(n)) ..
      + step21(n)-step21(n-4);

disp("(a) Values at n = -3:3:");
disp(signal21(-3:3));
disp("(b1) Values at n = 1:7:");
disp(signal21(4-(1:7)));
disp("(b2) Values at n = -7:-1:");
disp(signal21(-(-7:-1)-4));
disp("(d) Maximum difference between (b1) and (c), expected 0:");
disp(max(abs(yb1-yc)));
disp("(e) Impulse reconstruction error, expected approximately 0:");
disp(max(abs(xn-xdelta)));
disp("(e) Step reconstruction error, expected approximately 0:");
disp(max(abs(xn-xstep)));

// Four plots on the same time and amplitude scales.
signals = [xn; yb1; yb2; yc];
titles = ["(a) x(n)"; ..
          "(b1) Fold then delay: x(4-n)"; ..
          "(b2) Delay then fold: x(-n-4)"; ..
          "(c) x(-n+4)"];
scf();
for p = 1:4
    subplot(2, 2, p);
    plot2d3(n, signals(p, :));
    e = gce();
```

```
    e.children(1).thickness = 3;
    a = gca();
    a.data_bounds = [-8, -0.1; 8, 1.2];
    xtitle(titles(p), "n", "A");
    xgrid();
end
```

Listing 8.1: Scilab implementation of supplementary Problem 2.1

8.7 Execution evidence

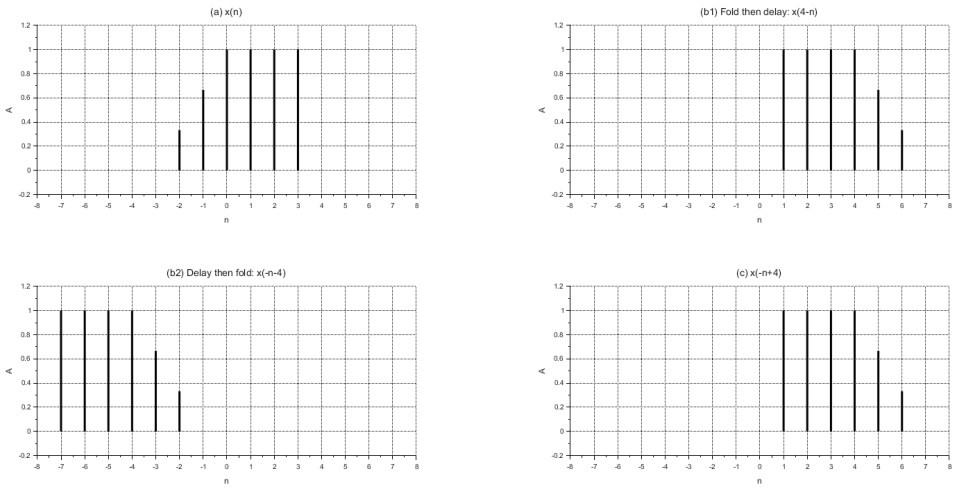


Figure 8.1: Original signal and the three transformations in Problem 2.1

The captured plots agree with the calculated sample values. In particular, parts (b1) and (c) coincide on the positive-time side, while part (b2) appears at the corresponding negative time indices. The console reconstruction errors are separate checks and are not shown in this figure.

9 Supplementary Problem 2.2

Figure P2.2 in the reference document specifies

$$x(n) = \begin{cases} 1, & n \in \{-1, 0, 1, 2\}, \\ \frac{1}{2}, & n \in \{3, 4\}, \\ 0, & \text{otherwise.} \end{cases} \tag{9.1}$$

The following results apply at integer time indices; all samples not listed in a result are zero. The unit-step convention is $u(0) = 1$.

9.1 Part (a): Delay by two samples

For $y_a(n) = x(n-2)$, every original time m maps to $m+2$:

$$y_a(n) = \begin{cases} 1, & n \in \{1, 2, 3, 4\}, \\ \frac{1}{2}, & n \in \{5, 6\}, \\ 0, & \text{otherwise.} \end{cases} \tag{9.2}$$

9.2 Part (b): Folding followed by a four-sample delay

For $y_b(n) = x(4-n)$, the sample mapping is $m \mapsto 4 - m$:

$$y_b(n) = \begin{cases} \frac{1}{2}, & n \in \{0, 1\}, \\ 1, & n \in \{2, 3, 4, 5\}, \\ 0, & \text{otherwise.} \end{cases} \tag{9.3}$$

9.3 Part (c): Advance by two samples

For $y_c(n) = x(n+2)$, every original time m maps to $m-2$:

$$y_c(n) = \begin{cases} 1, & n \in \{-3, -2, -1, 0\}, \\ \frac{1}{2}, & n \in \{1, 2\}, \\ 0, & \text{otherwise.} \end{cases} \tag{9.4}$$

9.4 Part (d): Multiplication by a reversed step

Since $u(2-n) = 1$ precisely when $n \leq 2$, the product retains the original samples up to and including $n = 2$:

$$y_d(n) = x(n)u(2-n) = \begin{cases} 1, & n \in \{-1, 0, 1, 2\}, \\ 0, & \text{otherwise.} \end{cases} \tag{9.5}$$

The samples of amplitude $1/2$ at $n = 3$ and $n = 4$ are removed.

9.5 Part (e): Multiplication by a shifted impulse

The impulse $\delta(n-3)$ selects only $n = 3$. Evaluating the other factor at this time gives $x(3-1) = x(2) = 1$, so

$$y_e(n) = x(n-1)\delta(n-3) = \delta(n-3). \tag{9.6}$$

The output has a single sample of amplitude one at $n = 3$.

9.6 Part (f): A squared time argument

The signal $y_f(n) = x(n^2)$ evaluates x at the time index n^2; it does not square the sample amplitudes. For integer n, the relevant square indices are $0, 1, 4$:

$$y_f(n) = \begin{cases} 1, & n \in \{-1, 0, 1\}, \\ \frac{1}{2}, & n \in \{-2, 2\}, \\ 0, & \text{otherwise.} \end{cases} \tag{9.7}$$

In particular, $n = \pm 2$ samples $x(4) = 1/2$, and $|n| \geq 3$ gives an argument at least nine, where x is zero.

9.7 Parts (g) and (h): Even and odd components

The required components are

$$x_e(n) = \frac{x(n) + x(-n)}{2}, \tag{9.8}$$

$$x_o(n) = \frac{x(n) - x(-n)}{2}. \tag{9.9}$$

They have the following sample values:

Table 9.1: Even and odd components of the signal in Figure P2.2

n	-4	-3	-2	-1	0	1	2	3	4
$x(n)$	0	0	0	1	1	1	1	1/2	1/2
$x(-n)$	1/2	1/2	1	1	1	1	0	0	0
$x_e(n)$	1/4	1/4	1/2	1	1	1	1/2	1/4	1/4
$x_o(n)$	−1/4	−1/4	−1/2	0	0	0	1/2	1/4	1/4

The even component satisfies $x_e(-n) = x_e(n)$, while the odd component satisfies $x_o(-n) = -x_o(n)$ and $x_o(0) = 0$. Adding the two components recovers the original signal at every time index.

9.8 Scilab implementation

The script evaluates the given signal at the required time arguments, uses elementwise products for parts (d) and (e), and uses `n.^2` to square each time index in part (f). Two figure windows show parts (a)–(d) and (e)–(h), respectively. All plots use the same time interval and amplitude limits. The script also prints the values for parts (f)–(h) and the reconstruction error $\max_n |x(n) - x_e(n) - x_o(n)|$, which is expected to be zero.

```
// Supplementary Problem 2.2: transformations of Figure P2.2.
function values = signal22(n)
    values = zeros(1, length(n));
    full = find((n >= -1) & (n <= 2));
    half = find((n == 3) | (n == 4));
    values(full) = 1;
    values(half) = 1/2;
endfunction

n = -5:7;
xn = signal22(n);
ya = signal22(n-2);
yb = signal22(4-n);
yc = signal22(n+2);
yd = xn .* bool2s(2-n >= 0);
ye = signal22(n-1) .* bool2s(n-3 == 0);
yf = signal22(n.^2);
```

```
xeven = (xn + signal22(-n))/2;
xodd = (xn - signal22(-n))/2;

// Window 1: parts (a)--(d).
signalsAD = [ya; yb; yc; yd];
titlesAD = ["(a) x(n-2)"; "(b) x(4-n)"; ..
            "(c) x(n+2)"; "(d) x(n)u(2-n)"];
scf();
for p = 1:4
    subplot(2, 2, p);
    plot2d3(n, signalsAD(p, :));
    e = gce();
    e.children(1).thickness = 3;
    a = gca();
    a.data_bounds = [-5, -0.6; 7, 1.2];
    xtitle(titlesAD(p), "n", "A");
    xgrid();
end

// Window 2: parts (e)--(h).
signalsEH = [ye; yf; xeven; xodd];
titlesEH = ["(e) x(n-1)delta(n-3)"; "(f) x(n^2)"; ..
            "(g) Even part of x(n)"; "(h) Odd part of x(n)"];
scf();
for p = 1:4
    subplot(2, 2, p);
    plot2d3(n, signalsEH(p, :));
    e = gce();
    e.children(1).thickness = 3;
    a = gca();
    a.data_bounds = [-5, -0.6; 7, 1.2];
    xtitle(titlesEH(p), "n", "A");
    xgrid();
end

disp("Input samples at n = -1:4:");
disp(signal22(-1:4));
disp("(f) x(n^2) at n = -2:2:");
disp(signal22((-2:2).^2));
nsym = -4:4;
disp("(g) Even part at n = -4:4:");
disp((signal22(nsym)+signal22(-nsym))/2);
disp("(h) Odd part at n = -4:4:");
```

```
disp((signal22(nsym)-signal22(-nsym))/2);
disp("Maximum reconstruction error, expected 0:");
disp(max(abs(xn-xeven-xodd)));
```

Listing 9.1: Scilab implementation of supplementary Problem 2.2

9.9 Execution evidence

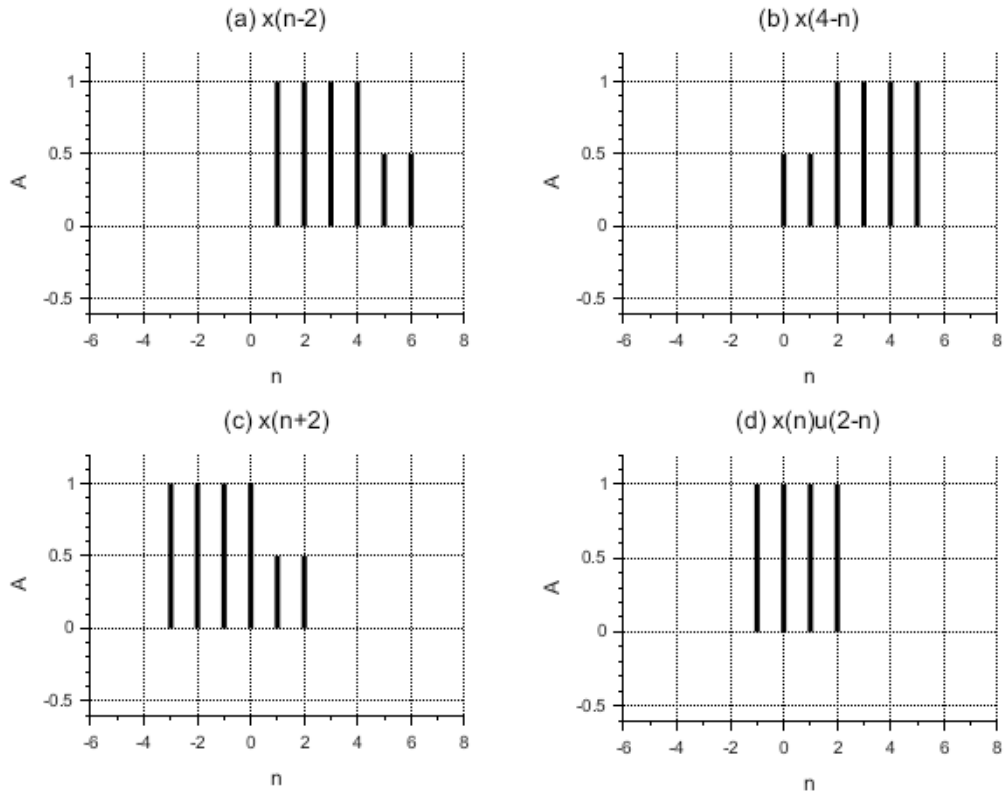


Figure 9.1: Time shifts, folding, and step multiplication in Problem 2.2(a)–(d)

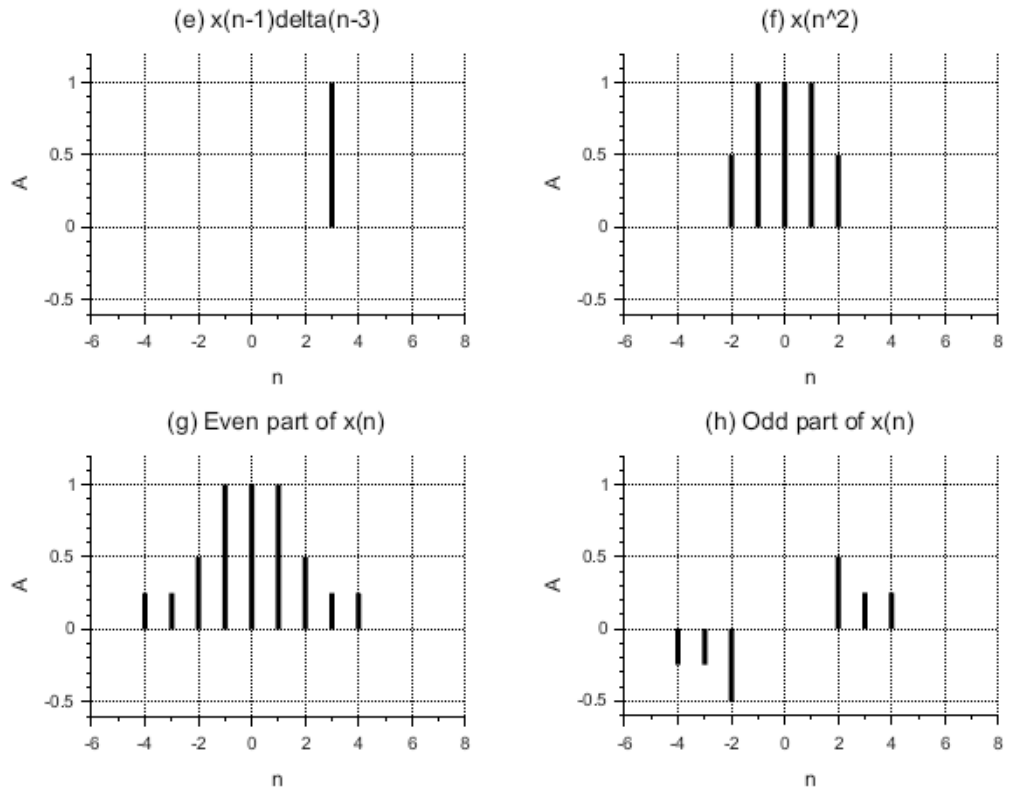


Figure 9.2: Impulse multiplication, squared time argument, and even and odd components in Problem 2.2(e)–(h)

The captured plots agree with the analytical results for all eight parts. In particular, part (e) contains one impulse at $n = 3$, part (f) has amplitudes $1/2, 1, 1, 1, 1/2$ at $n = -2, -1, 0, 1, 2$, and the even and odd components show the required symmetry. The numerical reconstruction error is a separate console check and is not displayed in these figures.

10 Supplementary Problem 2.4

The task is to show that every discrete-time signal can be decomposed into an even and an odd component, determine whether this decomposition is unique, and illustrate the result with $x(n) = \{2, 3, 4, 5, 6\}$, where the central sample 4 corresponds to $n = 0$.

10.1 Existence of the decomposition

For an arbitrary signal $x(n)$, define

$$x_e(n) = \frac{x(n) + x(-n)}{2}, \tag{10.1}$$

$$x_o(n) = \frac{x(n) - x(-n)}{2}. \tag{10.2}$$

Evaluating at $-n$ gives

$$x_e(-n) = \frac{x(-n) + x(n)}{2} = x_e(n), \tag{10.3}$$

$$x_o(-n) = \frac{x(-n) - x(n)}{2} = -x_o(n). \tag{10.4}$$

Thus x_e is even and x_o is odd. Their sum is

$$x_e(n) + x_o(n) = \frac{x(n) + x(-n) + x(n) - x(-n)}{2} = x(n), \tag{10.5}$$

which proves that the decomposition exists for every signal.

10.2 Uniqueness of the decomposition

Suppose $x(n) = a(n) + b(n)$ for some even signal a and odd signal b. Their symmetry implies

$$x(-n) = a(-n) + b(-n) = a(n) - b(n). \tag{10.6}$$

Adding and subtracting the two equations forces

$$a(n) = \frac{x(n) + x(-n)}{2} = x_e(n), \tag{10.7}$$

$$b(n) = \frac{x(n) - x(-n)}{2} = x_o(n). \tag{10.8}$$



Every even/odd decomposition therefore has exactly these components, so the decomposition is unique. This algebraic argument applies to arbitrary x; the numerical example below only illustrates the result.

10.3 Example and expected result

The given sequence is stored on $n = -2, -1, 0, 1, 2$, with zero values outside that interval. Since this time grid is symmetric about zero, reversing the vector gives the samples of $x(-n)$ on the same grid.

Table 10.1: Unique even and odd components for Problem 2.4

Time n	-2	-1	0	1	2
$x(n)$	2	3	4	5	6
$x(-n)$	6	5	4	3	2
$x_e(n)$	4	4	4	4	4
$x_o(n)$	-2	-1	0	1	2
$x_e(n) + x_o(n)$	2	3	4	5	6

Consequently,

$$x_e(n) = \begin{cases} 4, & |n| \le 2, \\ 0, & \text{otherwise}, \end{cases} \tag{10.9}$$

$$x_o(n) = \begin{cases} n, & |n| \le 2, \\ 0, & \text{otherwise}. \end{cases} \tag{10.10}$$

Both vectors retain origin position three. The even component is constant only on the stated finite interval, and the odd component has a zero sample at the origin as required.

10.4 Scilab implementation

The script computes the two components and their sum, then displays four plots: the original signal, the even component, the odd component, and the reconstructed signal. It

also prints three numerical checks:

$$\max_n |x(n) - x_e(n) - x_o(n)| = 0, \tag{10.11}$$

$$\max_n |x_e(n) - x_e(-n)| = 0, \tag{10.12}$$

$$\max_n |x_o(n) + x_o(-n)| = 0. \tag{10.13}$$

These are expected results for the specified example; the proof of uniqueness is the general derivation above.

```
// Supplementary Problem 2.4: unique even/odd decomposition.
n = -2:2;
xn = [2, 3, 4, 5, 6];
xorigin = 3;

// Reversal gives x(-n) here because n is symmetric about zero.
xfold = xn($:-1:1);
xeven = (xn + xfold)/2;
xodd = (xn - xfold)/2;
xreconstructed = xeven + xodd;

disp("Time indices n:");
disp(n);
disp("x(n):");
disp(xn);
disp("x(-n):");
disp(xfold);
disp("Even part:");
disp(xeven);
disp("Odd part:");
disp(xodd);
disp("Reconstruction error, expected 0:");
disp(max(abs(xn-xreconstructed)));
disp("Even symmetry error, expected 0:");
disp(max(abs(xeven-xeven($:-1:1))));
disp("Odd symmetry error, expected 0:");
disp(max(abs(xodd+xodd($:-1:1))));

signals = [xn; xeven; xodd; xreconstructed];
titles = ["Original x(n)"; "Even part xe(n)"; ..
          "Odd part xo(n)"; "Reconstruction xe(n) + xo(n)"];
scf();
for p = 1:4
```

```
    subplot(2, 2, p);
    plot2d3(n, signals(p, :));
    e = gce();
    e.children(1).thickness = 3;
    a = gca();
    a.data_bounds = [-3, -3; 3, 7];
    xtitle(titles(p), "n", "A");
    xgrid();
end
```

Listing 10.1: Scilab implementation of supplementary Problem 2.4

10.5 Execution evidence

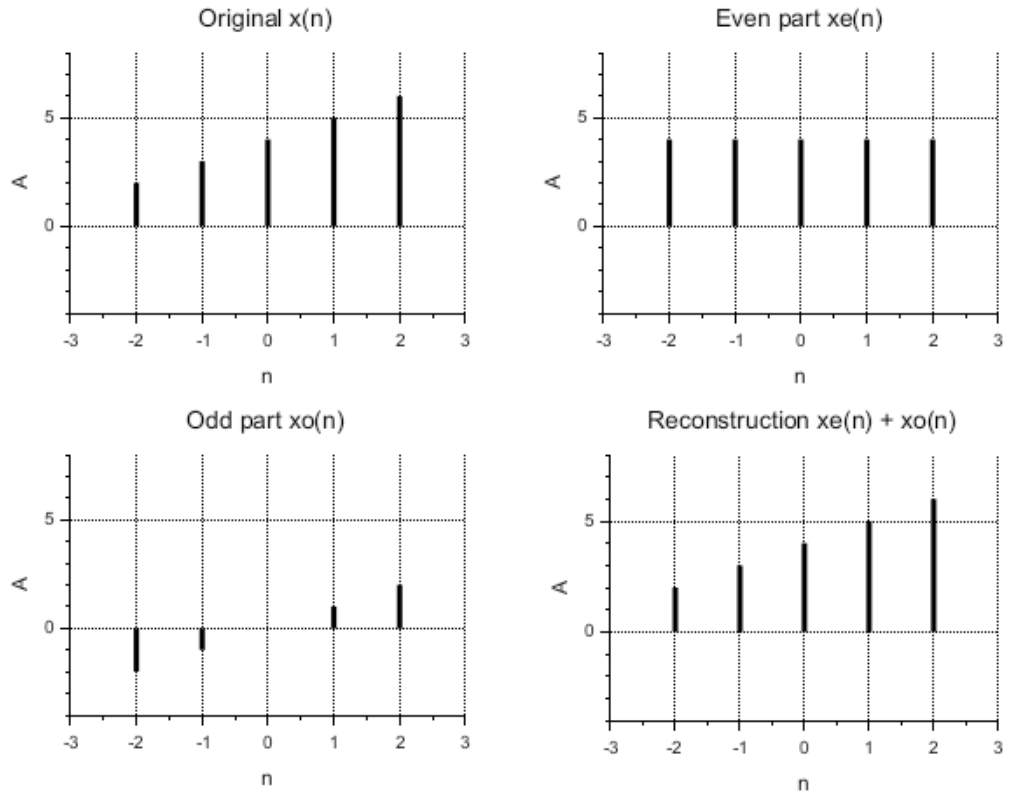


Figure 10.1: Original signal, even and odd components, and reconstruction for Problem 2.4

The captured plots match Table 10.1. The even component has five samples of amplitude four, the odd component has amplitudes $-2, -1, 0, 1, 2$, and the reconstructed signal matches the original.

11 Supplementary Problem 2.6

11.1 Criterion for time invariance

Let $y(n) = \mathcal{T}[x](n)$ and let $x_k(n) = x(n-k)$ be a delayed input. A system is time invariant if

$$\mathcal{T}[x_k](n) = y(n-k) \tag{11.1}$$

for every input x, integer shift k, and time n. The left side delays the input before applying the system; the right side delays the original output. A single counterexample disproves time invariance, whereas agreement for one example alone does not prove it.

11.2 Part (a): Squared-time system

For $\mathcal{T}[x](n) = x(n^2)$, the two paths give

$$\mathcal{T}[x_k](n) = x_k(n^2) = x(n^2 - k), \tag{11.2}$$

$$y(n-k) = x((n-k)^2). \tag{11.3}$$

These are not equal in general. For the pulse input used in part (b), choose $k = 2$ and $n = -2$. Then $x(n^2 - k) = x(2) = 1$, while $x((n-k)^2) = x(16) = 0$. This counterexample proves that the system is time varying.

11.3 Part (b): Explicit pulse example

The specified input is

$$x(n) = \begin{cases} 1, & 0 \le n \le 3, \\ 0, & \text{otherwise}, \end{cases} \qquad n \in \mathbb{Z}. \tag{11.4}$$

Use $k = 2$ and write $x_2(n) = x(n-2)$, $y_2'(n) = y(n-2)$, and $y_2(n) = \mathcal{T}[x_2](n)$. The five signals requested in steps (1)–(5) are determined as follows; all unlisted samples are zero.

Table 11.1: Pulse example for the squared-time system

Signal	Nonzero time indices	Sample values
$x(n)$	$0, 1, 2, 3$	$1, 1, 1, 1$
$y(n) = x(n^2)$	$-1, 0, 1$	$1, 1, 1$
$y_2'(n) = x((n-2)^2)$	$1, 2, 3$	$1, 1, 1$
$x_2(n) = x(n-2)$	$2, 3, 4, 5$	$1, 1, 1, 1$
$y_2(n) = x(n^2-2)$	$-2, 2$	$1, 1$

For the original output, $0 \le n^2 \le 3$ gives $n = -1, 0, 1$. For the shifted-input output, $0 \le n^2 - 2 \le 3$ gives $2 \le n^2 \le 5$, whose only integer solutions are $n = \pm 2$. For step (6), the discrepancy is

$$y_2(n) - y_2'(n) = \delta(n+2) - \delta(n-1) - \delta(n-3). \tag{11.5}$$

The two paths differ, confirming the conclusion in part (a).

11.4 Part (c): First-difference system

Now consider $\mathcal{T}[x](n) = x(n) - x(n-1)$. For an arbitrary input and arbitrary integer delay,

$$\mathcal{T}[x_k](n) = x(n-k) - x(n-1-k), \tag{11.6}$$

$$y(n-k) = x(n-k) - x(n-k-1). \tag{11.7}$$

The two expressions are identical for all x, n, k. Hence this system is time invariant.

For the same pulse input, the original output has one positive sample at the leading edge and one negative sample immediately after the pulse:

$$y(n) = \delta(n) - \delta(n-4). \tag{11.8}$$

Repeating steps (1)–(5) gives the following nonzero samples:

Table 11.2: Pulse example for the first-difference system

Signal	Nonzero time indices	Sample values
$x(n)$	$0, 1, 2, 3$	$1, 1, 1, 1$
$y(n)$	$0, 4$	$1, -1$
$y_2'(n) = y(n-2)$	$2, 6$	$1, -1$
$x_2(n)$	$2, 3, 4, 5$	$1, 1, 1, 1$
$y_2(n) = \mathcal{T}[x_2](n)$	$2, 6$	$1, -1$

Thus $y_2(n) = y_2'(n) = \delta(n-2) - \delta(n-6)$ and their difference is zero. This particular agreement illustrates time invariance; the general algebraic identity above is what proves it.

11.5 Part (d): Multiplication by time

For $\mathcal{T}[x](n) = nx(n)$, the two paths are

$$\mathcal{T}[x_k](n) = nx(n-k), \tag{11.9}$$

$$y(n-k) = (n-k)x(n-k). \tag{11.10}$$

Their difference is $kx(n-k)$, which is nonzero for suitable inputs and nonzero k. Consequently this system is time varying.

For the given pulse and $k = 2$, the nonzero samples in steps (1)–(5) are

Table 11.3: Pulse example for the time-multiplication system

Signal	Nonzero time indices	Sample values
$x(n)$	$0, 1, 2, 3$	$1, 1, 1, 1$
$y(n) = nx(n)$	$1, 2, 3$	$1, 2, 3$
$y_2'(n) = (n-2)x(n-2)$	$3, 4, 5$	$1, 2, 3$
$x_2(n)$	$2, 3, 4, 5$	$1, 1, 1, 1$
$y_2(n) = nx(n-2)$	$2, 3, 4, 5$	$2, 3, 4, 5$

The difference $y_2(n) - y_2'(n) = 2x(n-2)$ equals two at $n = 2, 3, 4, 5$ and zero elsewhere.



For example, at $n = 2$ the shifted output is zero while the response to the shifted input is two.

11.6 Scilab implementation and expected results

The script uses a direct evaluator for the pulse to avoid truncating shifted signals at vector boundaries. It creates one figure per system, each containing the five requested signals and a sixth plot of $\mathcal{T}[x_2](n) - y(n-2)$. The interval $-4 \le n \le 8$ contains all nonzero samples for these examples. The maximum absolute discrepancies are expected to be 1, 0, and 2 for parts (b), (c), and (d), respectively.

```
// Supplementary Problem 2.6: time invariance, input delay k = 2.
function values = signal26(n)
    values = bool2s((n >= 0) & (n <= 3));
endfunction

function plot26(n, xn, yn, ydelayed, xdelayed, yshiftinput, systemTitle)
    difference = yshiftinput - ydelayed;
    signals = [xn; yn; ydelayed; xdelayed; yshiftinput; difference];
    titles = ["(1) x(n)"; "(2) y(n) = T[x(n)]"; ..
              "(3) Shift output: y(n-2)"; "(4) Shift input: x(n-2)"; ..
              "(5) T[x(n-2)]"; "(6) T[x(n-2)] - y(n-2)"];
    ymin = min([0, xn, yn, ydelayed, xdelayed, yshiftinput, difference])-1;
    ymax = max([0, xn, yn, ydelayed, xdelayed, yshiftinput, difference])+1;

    scf();
    f = gcf();
    f.figure_name = systemTitle;
    f.figure_size = [1200, 800];
    for p = 1:6
        subplot(3, 2, p);
        plot2d3(n, signals(p, :));
        e = gce();
        e.children(1).thickness = 3;
        a = gca();
        a.data_bounds = [n(1), ymin; n($), ymax];
        xtitle(systemTitle + " | " + titles(p), "n", "A");
        xgrid();
    end
    disp(systemTitle);
    disp("Maximum absolute difference on the displayed interval:");
    disp(max(abs(difference)));
```

```
endfunction

n = -4:8;
k = 2;
xn = signal26(n);
xdelayed = signal26(n-k);

// (a), (b): T[x](n) = x(n^2).
yB = signal26(n.^2);
yBdelayed = signal26((n-k).^2);
yBshiftinput = signal26(n.^2-k);
plot26(n, xn, yB, yBdelayed, xdelayed, yBshiftinput, "(b) x(n^2)");

// (c): T[x](n) = x(n)-x(n-1).
yC = signal26(n)-signal26(n-1);
yCdelayed = signal26(n-k)-signal26(n-k-1);
yCshiftinput = xdelayed-signal26((n-1)-k);
plot26(n, xn, yC, yCdelayed, xdelayed, yCshiftinput, "(c) x(n)-x(n-1)");

// (d): T[x](n) = n*x(n).
yD = n.*xn;
yDdelayed = (n-k).*signal26(n-k);
yDshiftinput = n.*xdelayed;
plot26(n, xn, yD, yDdelayed, xdelayed, yDshiftinput, "(d) n*x(n)");

// Expected differences: (b) 1, (c) 0, (d) 2.
// One counterexample disproves time invariance. Agreement for one input
// does not prove it; use the general derivation in the report for (c).
```

Listing 11.1: Scilab implementation of supplementary Problem 2.6

11.7 Execution evidence

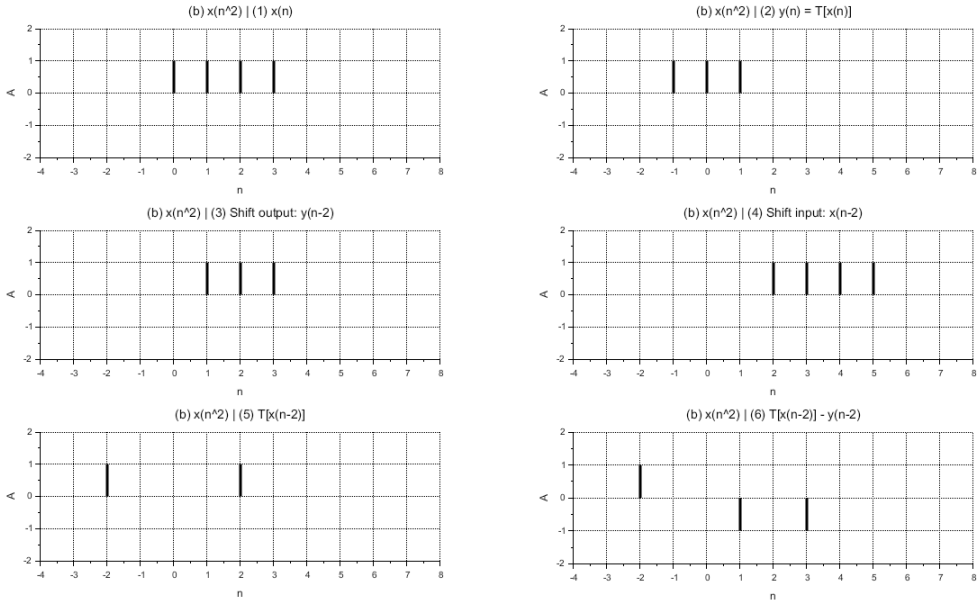


Figure 11.1: Pulse experiment for the squared-time system in Problem 2.6(b)

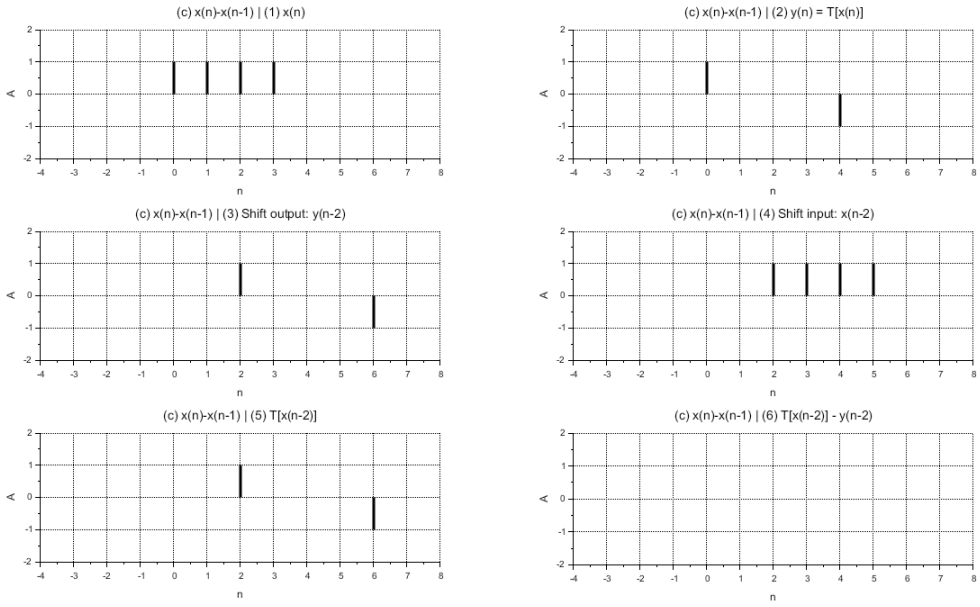


Figure 11.2: Pulse experiment for the first-difference system in Problem 2.6(c)

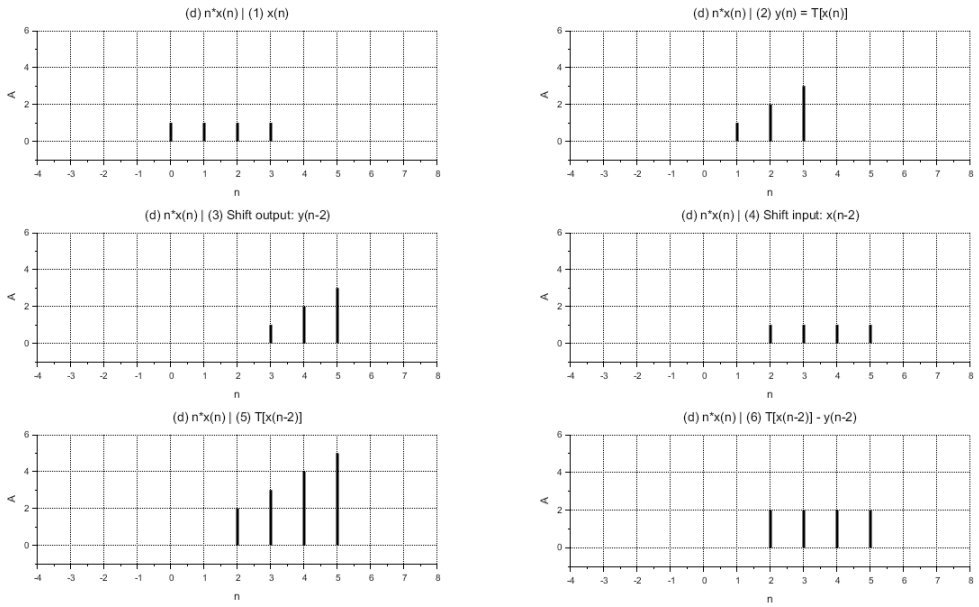


Figure 11.3: Pulse experiment for the time-multiplication system in Problem 2.6(d)

12 Conclusion

Representing a discrete-time signal requires both its sample vector and its time origin. Time shifts change the origin, folding reverses the sample order and updates the origin, and addition or pointwise multiplication requires alignment by time. Convolution accumulates products at sums of input time indices and has a different origin rule.

The supplementary derivations establish that folding and shifting do not generally commute and that the decomposition into even and odd components exists and is unique. For the three systems in Problem 2.6, the first-difference system is time invariant, while the squared-time and time-multiplication systems are time varying. A failed shift test provides a counterexample; proving time invariance requires an identity valid for arbitrary inputs and shifts.